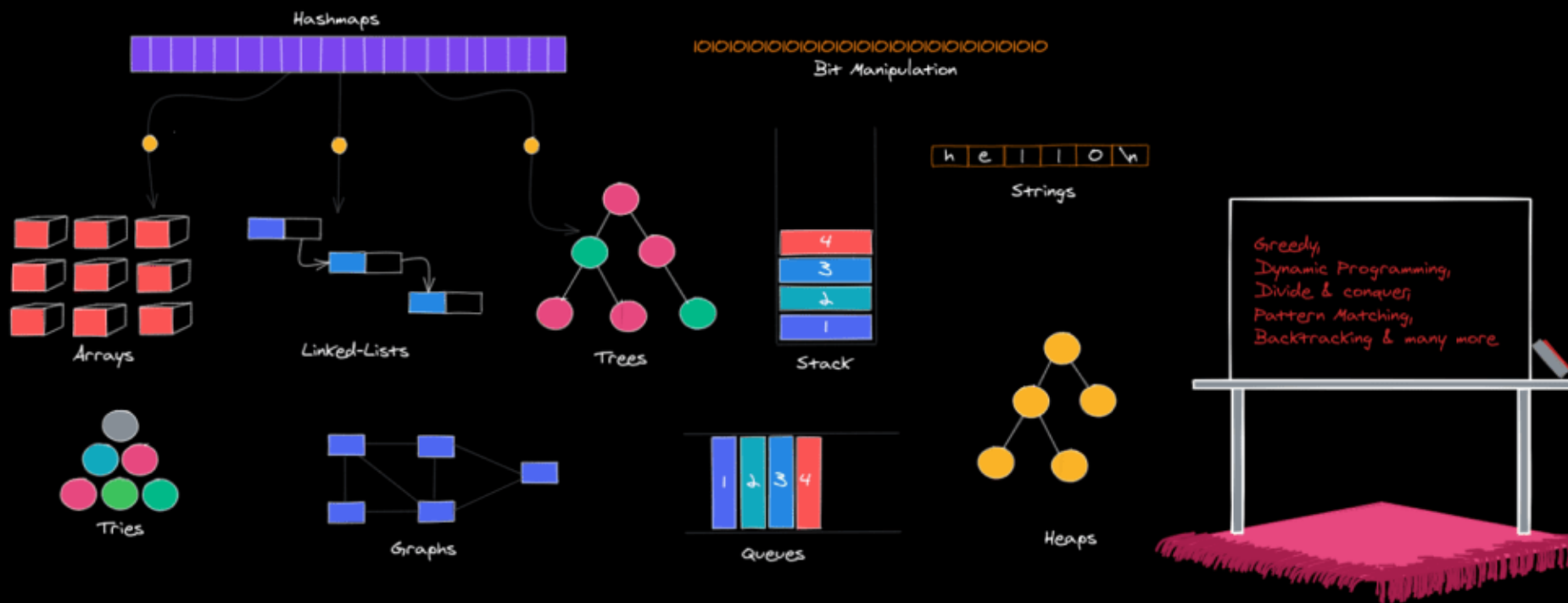




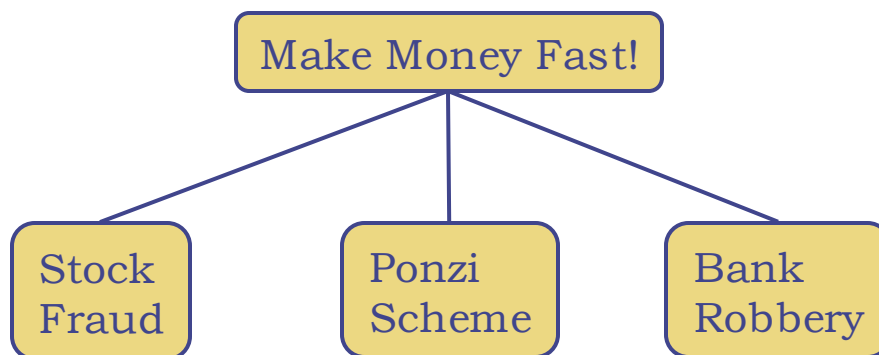
数据结构与算法导论

刘子博

FDSM

liuzibo@fudan.edu.cn

树



Today's Agenda

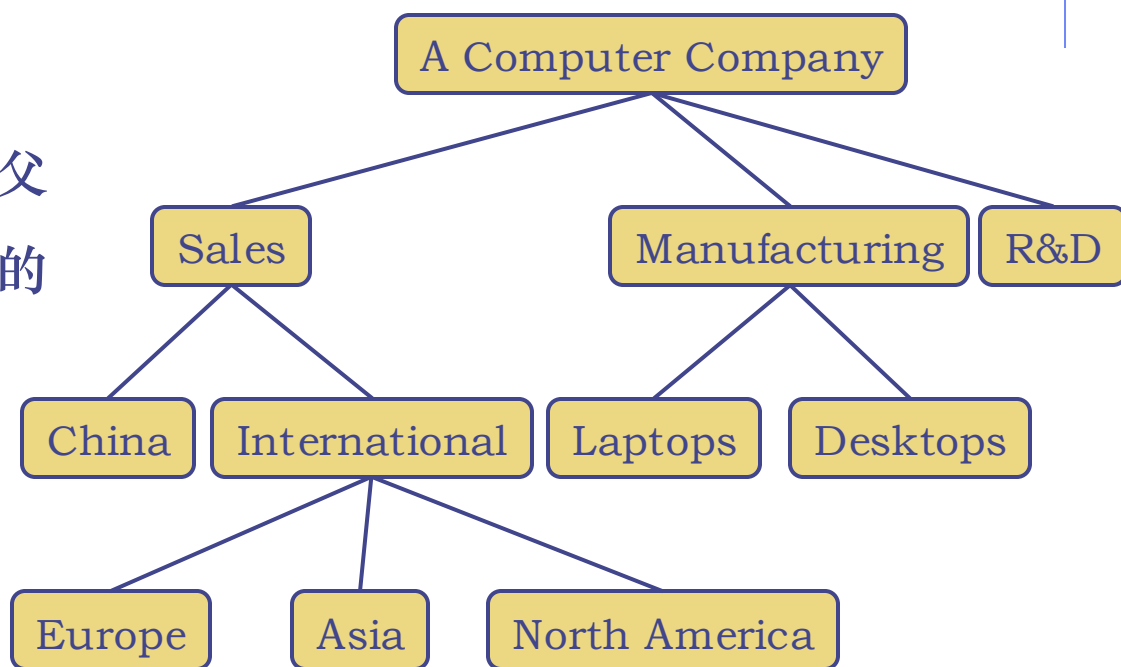
- 树
- 二叉树
- 树的实现
- 树的遍历算法

Today's Agenda

- 树
- 二叉树
- 树的实现
- 树的遍历算法

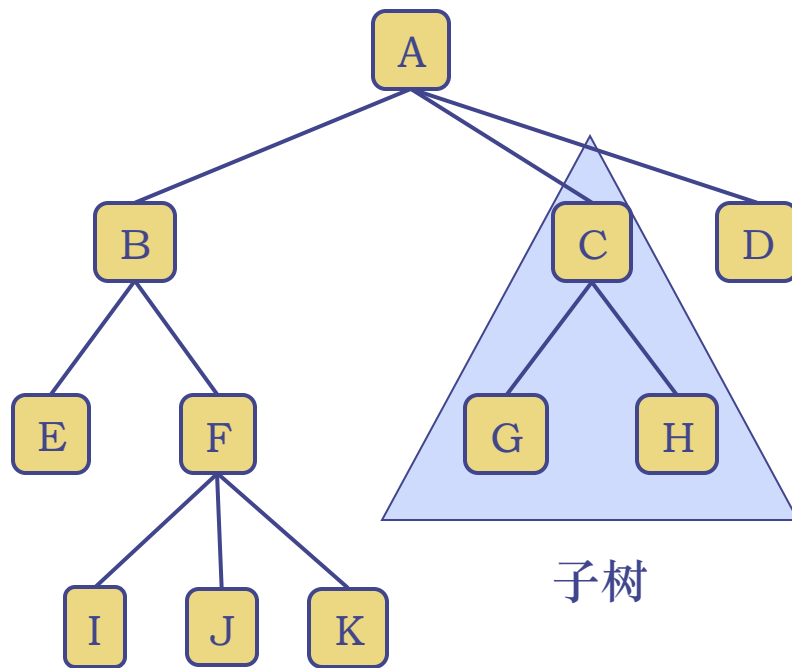
树

- 树是一种具有层次结构的抽象模型
- 树由节点和节点间表示父节点和孩子节点间关系的连线构成
- 树的应用：
 - 组织结构图
 - 文件系统
 - 排序算法
 - 搜索结构



树的定义

- **根节点 (root)** : 只有直接后继, 没有先驱节点 (A)
- 根以外的节点有且仅有唯一的先驱节点
- 若树的节点数目大于1, 则除根节点外的节点可以分成 m ($m > 0$) 个不相交的有限子集, 每个子集是一棵树, 并称为根节点的**子树 (subtree)**



树的有关术语

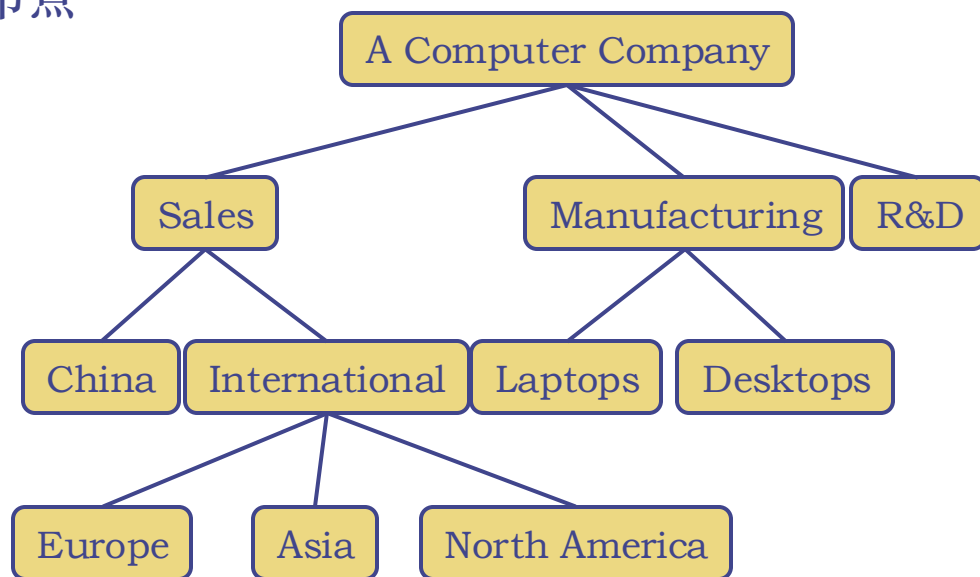
- **根节点 (root)** : 只有直接后继, 没有先驱节点
- **节点 (node)** : 树的基本组成单位, 包含一个数据元素和指向子树的分支
- **节点的度 (degree)** : 每个节点拥有的子树的数目
- **叶子节点 (leaf)** : 度为0的节点, 也称为外部节点 (external node)
- **分支节点**: 度不为0的节点, 也称为内部节点 (internal node)
- **孩子节点 (child)** : 某个节点的直接后继
- **父节点 (parent)** : 一个节点是其孩子节点的父节点, 也称为双亲节点
- **兄弟节点 (sibling)** : 具有相同父节点的节点
- **边 (edge)** : 一对节点 (u, v) , u 是 v 的父节点或反之
- **路径 (path)** : 一条从 n_i 到 n_j 的路径 $n_i, n_{i+1}, \dots, n_j$ 中任意两个相邻的节点前者是后者的父节点 (形成一条边)

树的有关术语

- **祖先节点 (ancestor)** : 根节点到某节点的路径上所有的节点都是该节点的祖先节点
- **子孙节点 (descendant)** : 某节点的所有子树上的节点都是其子孙节点
- **树的度**: 树中节点度的最大值
- **节点的深度 (depth)** : 节点的祖先节点的个数, 也称为**层次 (level)**
- **节点的高度 (height)** : 节点的高度是连接它与它的子孙节点中叶子节点的最长路径的长度
- **树的高度**: 根节点的高度为树的高度
- **有序树 (无序树)** : 树中节点的子树从左到右安排有序 (无序), 各子树位置不能 (可以) 交换
- **森林 (forest)** : 数棵互不相交的树的集合

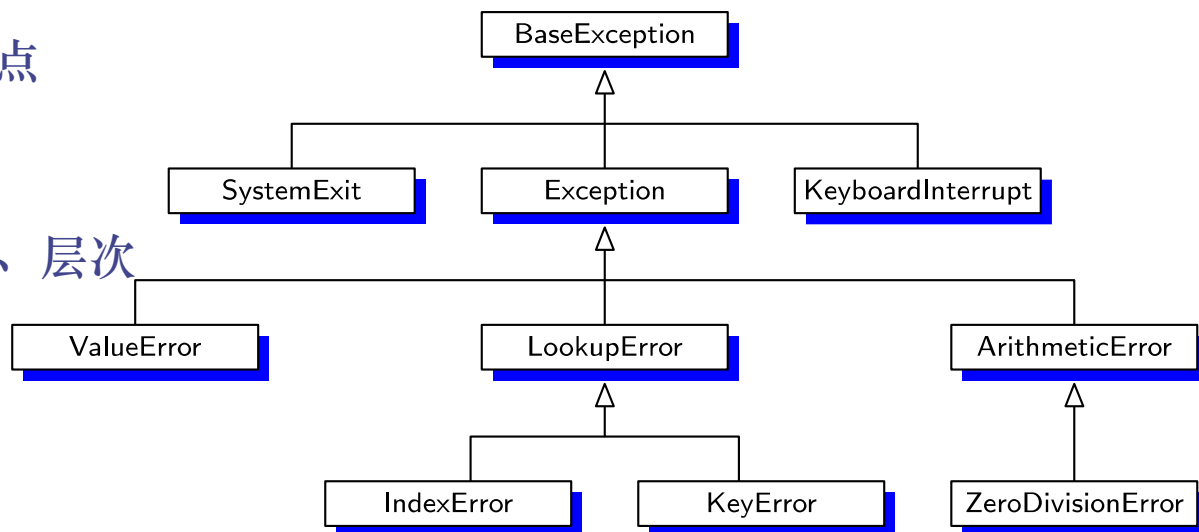
树的例子

- 根节点、分支节点、叶子节点
- 节点的度、树的度
- 父节点、孩子节点、兄弟节点
- 祖先节点、子孙节点
- 边、路径
- 节点的深度、高度、层次
- 树的高度



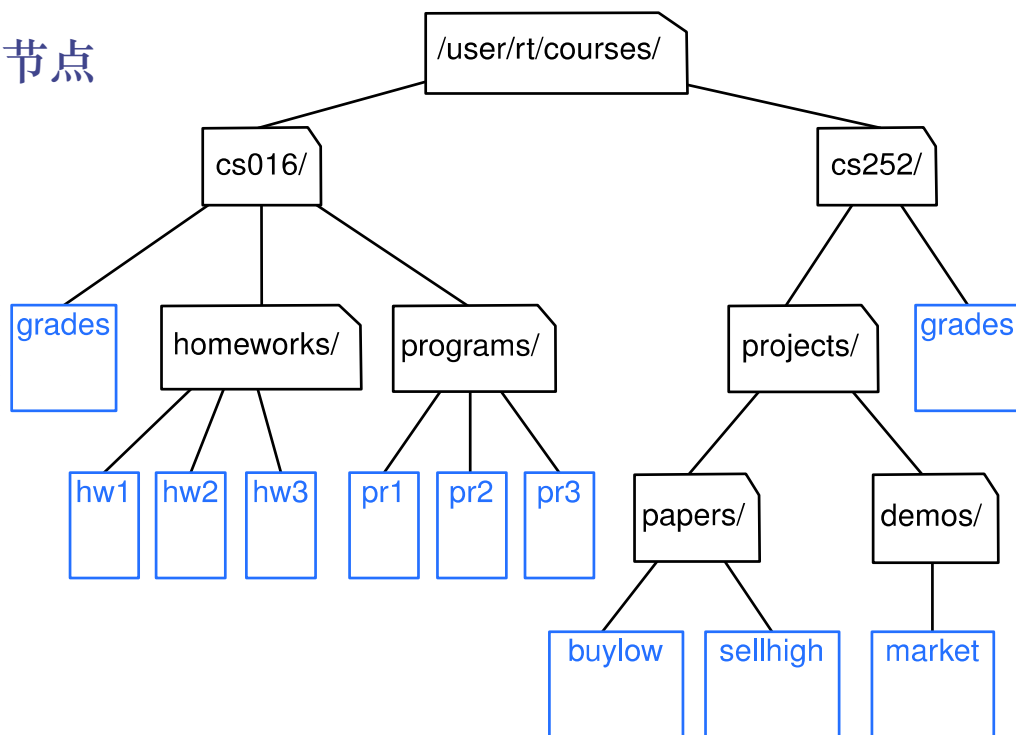
树的例子

- 根节点、分支节点、叶子节点
- 节点的度、树的度
- 父节点、孩子节点、兄弟节点
- 祖先节点、子孙节点
- 边、路径
- 节点的深度、高度、层次
- 树的高度



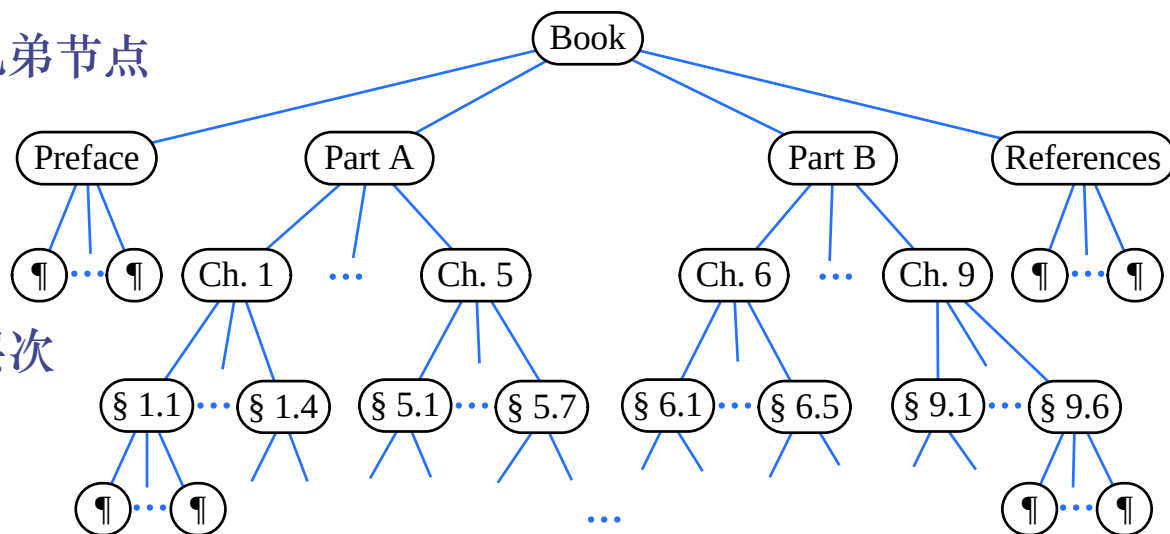
树的例子

- 根节点、分支节点、叶子节点
- 节点的度、树的度
- 父节点、孩子节点、兄弟节点
- 祖先节点、子孙节点
- 边、路径
- 节点的深度、高度、层次
- 树的高度



树的例子

- 根节点、分支节点、叶子节点
- 节点的度、树的度
- 父节点、孩子节点、兄弟节点
- 祖先节点、子孙节点
- 边、路径
- 节点的深度、高度、层次
- 树的高度
- 有序树

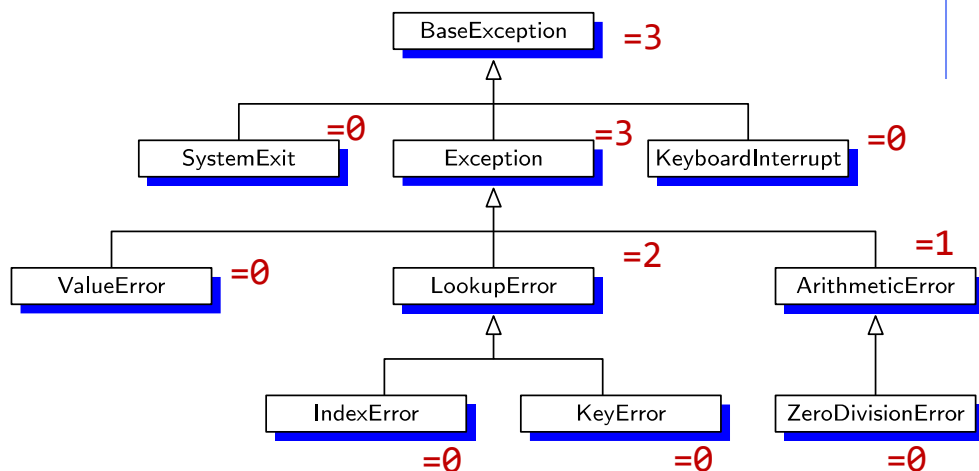


树的性质

- 1. 树的节点数等于所有节点度的总和加1

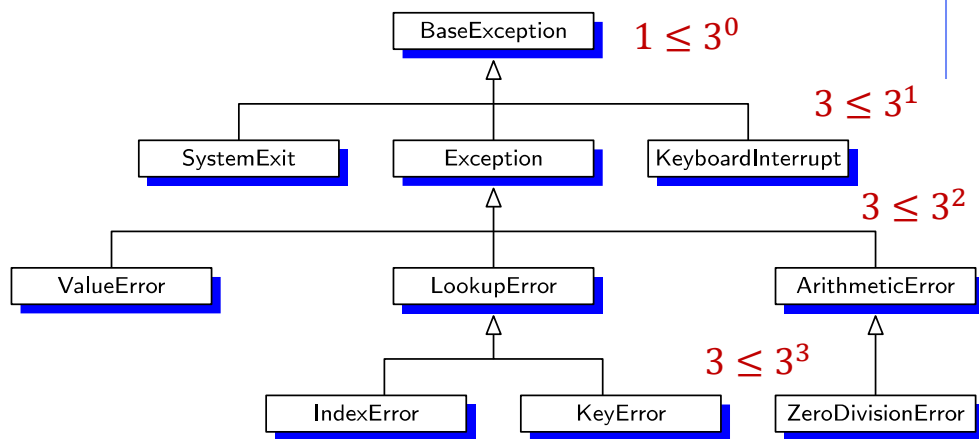
- 例: $\sum_{i=1}^n d_i = 9, n = 10$
- 节点的度等于其后继节点的数目
- 除根节点外, 每个节点有一个分支指向
- 总的分支数目等于除根节点外所有节点的数目
- 考虑根节点, 则有:

$$n = \sum_{i=1}^n d_i + 1$$



树的性质

- 2. 度为 k 的树，第 i 层至多有 k^i 个节点
 - 第0层仅有根节点， $k^0 = 1$
 - 假设结论对第 $i-1$ 层成立，即该层最多有 k^{i-1} 个节点
 - 树的度为节点度的最大值，即第 $i-1$ 层每个节点最多有 k 个后继
 - 因此第 i 层至多有 $k \times k^{i-1} = k^i$ 个节点



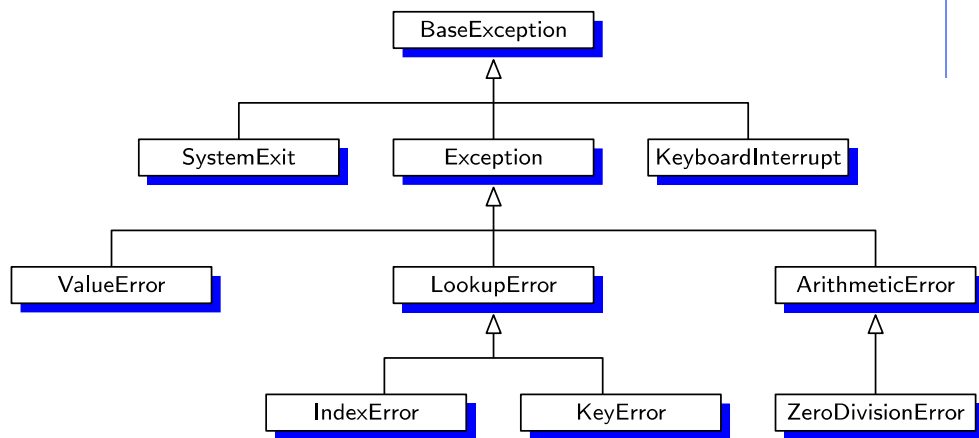
树的性质

- 3. 度为 k 且高度为 h 的树，至

多有 $\frac{k^{h+1}-1}{k-1}$ 个节点

- 根据性质2，第 i 层至多有 k^i 个节点
- 则总共的节点数不超过 $k^0 + k^1 + \dots + k^h$
- 即总节点数不超过：

$$\sum_{i=0}^h k^i = \frac{k^{h+1} - 1}{k - 1}$$



$$10 \leq \frac{3^4 - 1}{3 - 1} = 40$$

树的性质

- 4. 有 n 个节点的、度为 k 的树的最小高度为 $\lceil \log_k(n(k-1) + 1) \rceil - 1$
- 要想达到最小高度，则第0、1、 $\dots$ 、 $h-1$ 层均需尽可能多地存储节点

- 由性质3可得：

$$\frac{k^h - 1}{k - 1} < n \leq \frac{k^{(h+1)} - 1}{k - 1}$$

- 变换可得：

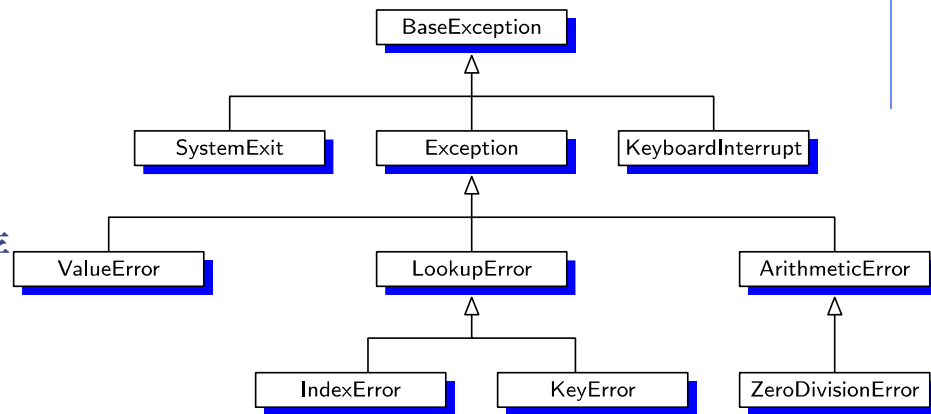
$$k^h < n(k-1) + 1 \leq k^{h+1}$$

- 取对数：

$$h < \log_k(n(k-1) + 1) \leq h + 1$$

- 因此有：

$$h_{\min} = \lceil \log_k(n(k-1) + 1) \rceil - 1$$



$$3 \geq \lceil \log_3(10 \times (3-1) + 1) \rceil - 1 = 2$$

树的ADT

- 树中的节点ADT仅支持一个方法：
 - `p.element()`: 返回存储在节点p处的元素

ADT Tree{

数据对象: T

基本操作:

`T.root()`: 返回树T的根节点, 如T为空则返回None

`T.parent(p)`: 返回p的父节点, 如p为根节点则返回None

`T.children(p)`: 产生p的孩子节点的迭代器

`T.num_children(p)`: 返回p的孩子节点的数目

树的ADT

`T.is_root(p)`: 如`p`为`T`的根, 则返回`True`

`T.is_leaf(p)`: 如`p`为树`T`的叶子节点, 则返回`True`

`T.depth(p)`: 返回树`T`中`p`节点的深度

`T.height(p)`: 返回树`T`中`p`节点的高度

`T.is_empty()`: 如`T`为空树则返回`True`

`len(T)`: 返回树`T`中元素 (节点) 个数

`T.nodes()`: 生成树`T`所有节点的迭代器

`iter(T)`: 生成树`T`中存储的所有元素的迭代

} ADT Tree

- 不同的树会附加不同的操作

树的python实现

- 树的部分方法的python实现:

```
def is_root(self, p):  
    """Return True if Node p represents the root of the tree."""  
    return self.root() is p  
  
def is_leaf(self, p):  
    """Return True if Node p does not have any children."""  
    return self.num_children(p) == 0  
  
def is_empty(self):  
    """Return True if the tree is empty."""  
    return len(self) == 0  
  
def __iter__(self):  
    """Generate an iteration of the tree's elements."""  
    for p in self.nodes():  
        yield p.element()  
    # use same order as nodes()  
    # but yield each element  
  
def nodes(self):  
    """Generate an iteration of the tree's nodes."""  
    return self.preorder() # return entire preorder iteration
```

树的深度

- 根据树中节点的深度的定义，节点的祖先节点的个数即为深度，故可以使用递归的方式计算深度：
 - 如果p为根节点，则其深度为0
 - 否则，p的深度为其父节点的深度加1
- 在Python中计算树中节点的深度：

```
def depth(self, p):  
    """Return the number of levels separating Node p from the root."""  
    if self.is_root(p):  
        return 0  
    else:  
        return 1 + self.depth(self.parent(p))
```

- 时间复杂度？
 - 与节点深度相关，一般情况下远小于节点总数n

树的高度

- 根据树中节点的高度的定义，连接它与它的子孙节点中叶子节点的最长路径的长度即为高度，故可以使用递归的方式计算高度：
 - 如果p为叶子节点，则其高度为0
 - 否则，p的高度为其孩子节点中最大的高度加1
- 根节点的高度即为树的**高度**

```
def height(self, p):                                # works and runs really fast
    """Return the height of the subtree rooted at Node p."""
    if self.is_leaf(p):
        return 0
    else:
        return 1 + max(self.height(c) for c in self.children(n))
```

- 时间复杂度？
 - 考虑递归调用将进行多少次
 - 如以p为根节点的子树有 n_p 个节点，则时间复杂度为 $O(n_p)$



Today's Agenda

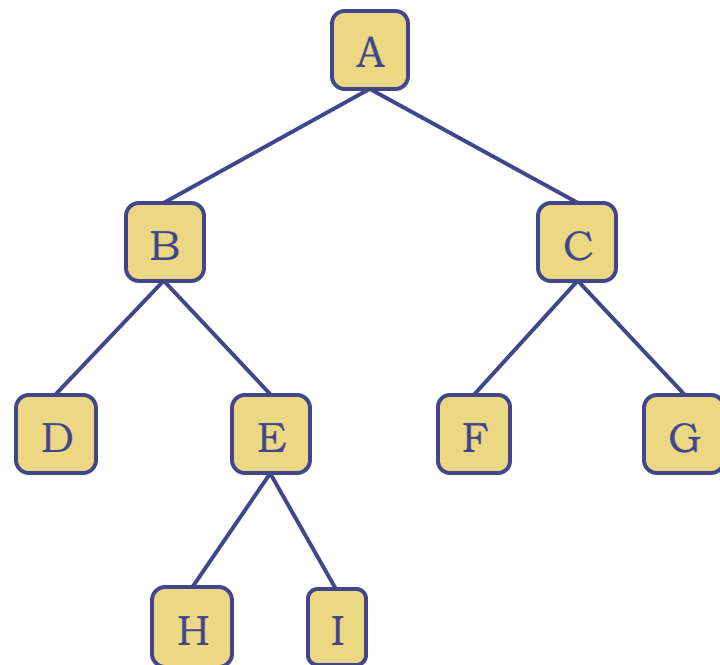
- 树
- 二叉树
- 树的实现
- 树的遍历算法

二叉树

- 一棵二叉树是一棵满足以下性质的树：
 - 每个内部节点最多有两个孩子节点
 - 孩子节点是有序的
- 内部节点的孩子节点被称为**左孩子**和**右孩子**，对应的子树被称为**左子树**和**右子树**
- 也可以**递归**定义二叉树，二叉树或为空树，或满足以下任一条件：
 - 为一个单独的节点
 - 根节点有至多两个有序的孩子节点，且根节点的子树也为二叉树

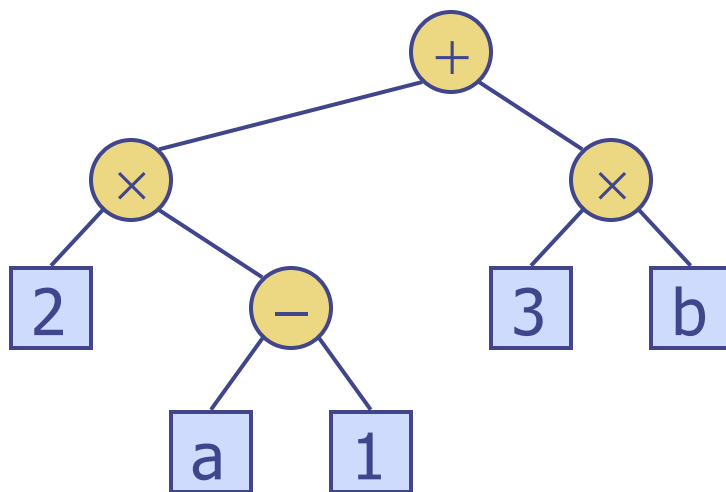
• 二叉树的应用

- 算术表达式
- 决策树
- 搜索树



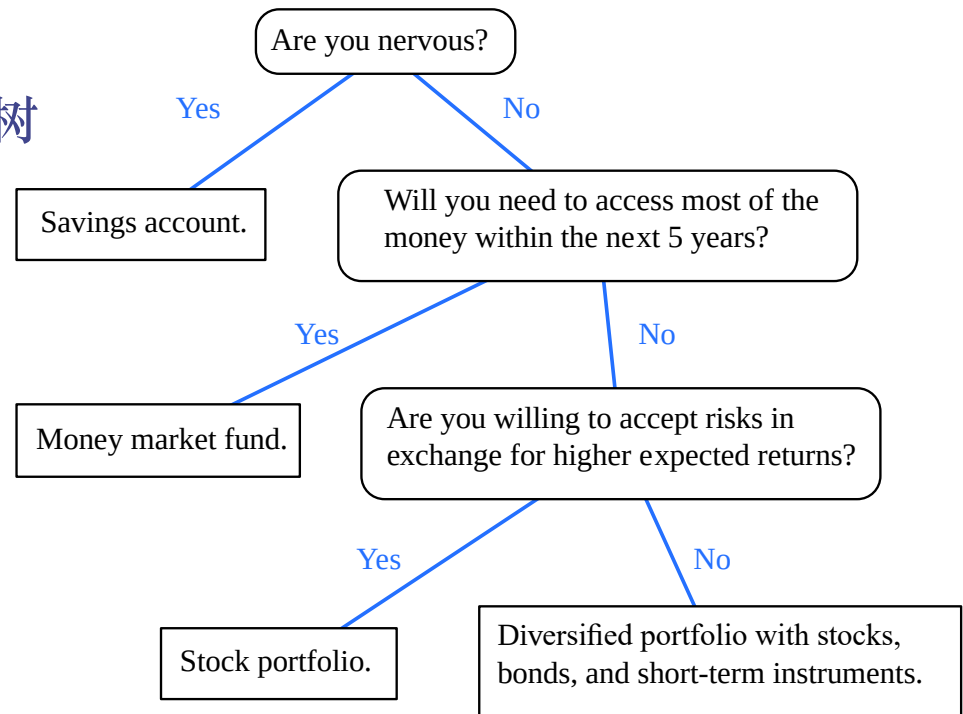
算术表达式树

- 二叉树能用于表示算术表达式
 - 内部节点：存储操作符
 - 叶子节点：存储变量或常数
- 例：表达式 $(2 \times (a - 1) + (3 \times b))$ 对应的算术表达式树



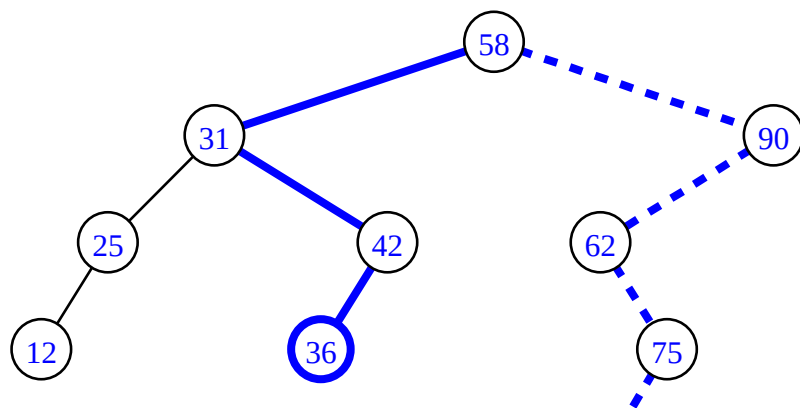
决策树

- 二叉树可以用来描述决策过程
 - 内部节点：存储一个回答为“是”或“否”的问题
 - 叶子节点：存储决策
- 例：提供投资决策的决策树



搜索树

- 二叉树可存储有序序列，并使用此二叉树进行搜索
 - 节点：存储元素
 - 节点左子树（右子树）的元素均小于（大于）节点存储的元素
- 例：实线为搜索36（成功），虚线为搜索70（不成功）

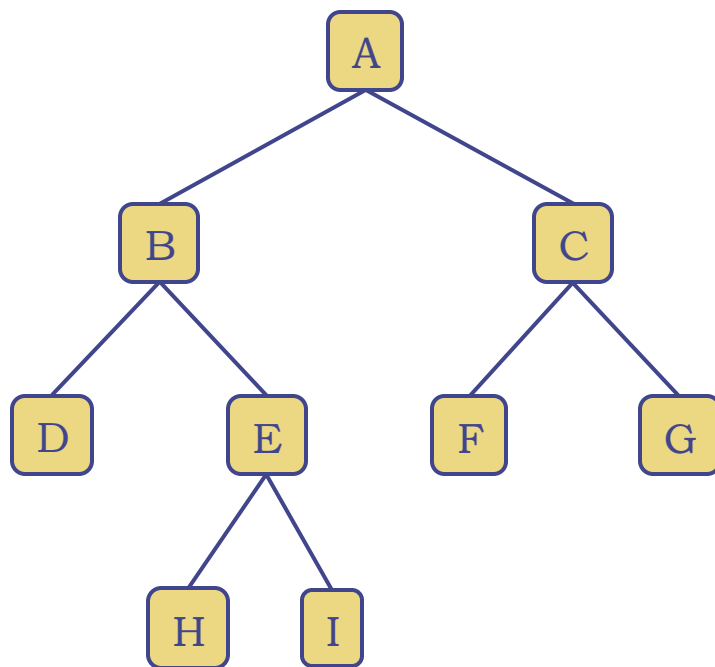


二叉树的性质

- 二叉树的一些性质可由树的性质直接推导得到
- 一些符号：
 - n : 节点数
 - n_E : 叶子节点数
 - n_I : 内部节点数
 - h : 树的高度

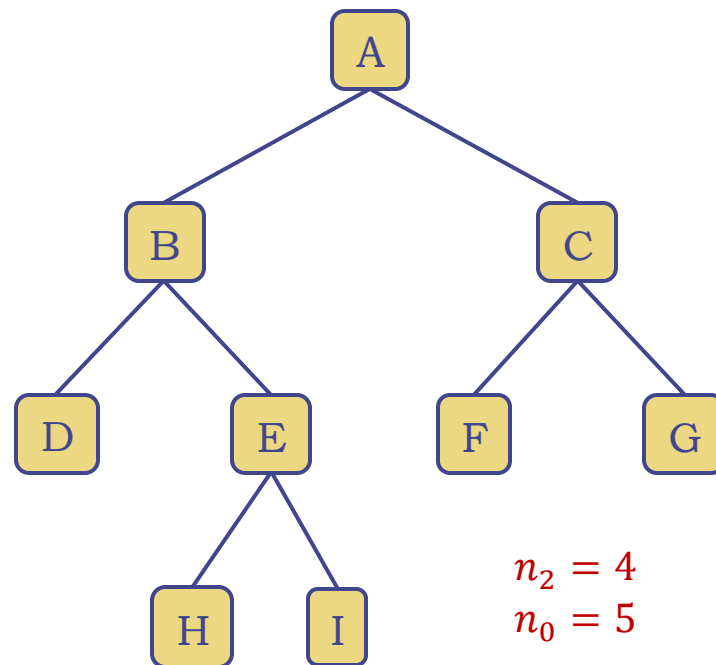
二叉树的性质

- 1. 第 i 层至多有 2^i 个节点
- 2. $h + 1 \leq n \leq 2^{(h+1)} - 1$
- 3. $h_{min} = \lceil \log(n + 1) \rceil - 1$
- 4. $1 \leq n_E \leq 2^h$
- 5. $h \leq n_I \leq 2^h - 1$



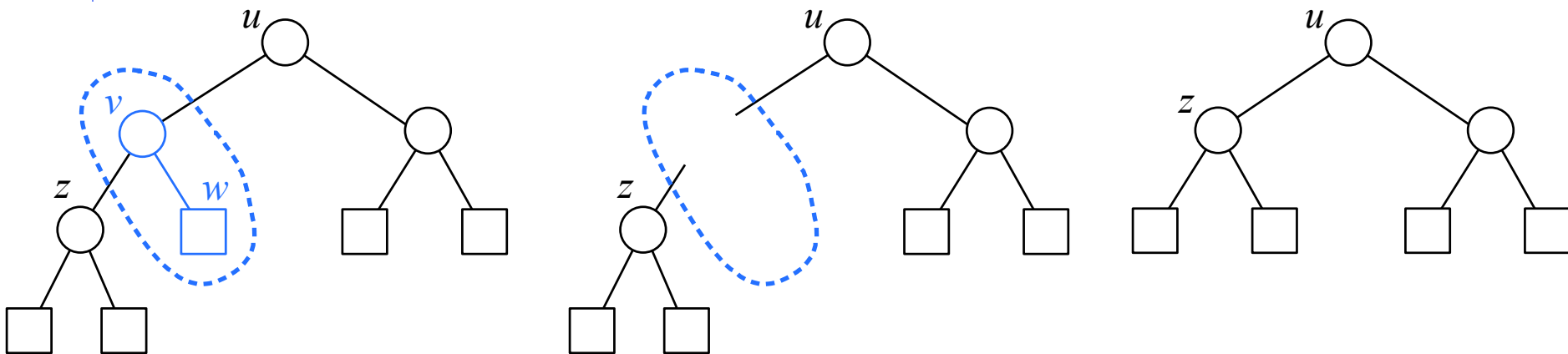
二叉树的性质

- 6. 对任何非空二叉树，设叶子节点数为 n_0 ，度为2的节点数为 n_2 ，则有： $n_0 = n_2 + 1$
 - 设二叉树中度为1度节点个数为 n_1 ，则有 $n = n_0 + n_1 + n_2$
 - 根据树的性质1，有 $n = n_1 + 2n_2 + 1$
 - 比较以上两式，可得 $n_0 = n_2 + 1$
 - 可否使用数学归纳法证明？



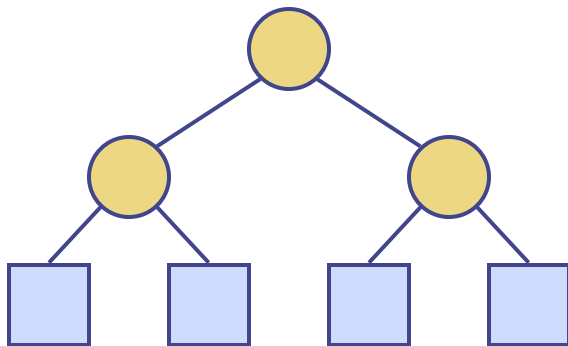
二叉树的性质

- 6. 对任何非空二叉树，设叶子节点数为 n_0 ，度为2的节点数为 n_2 ，
则有： $n_0 = n_2 + 1$
- 可否使用数学归纳法证明？

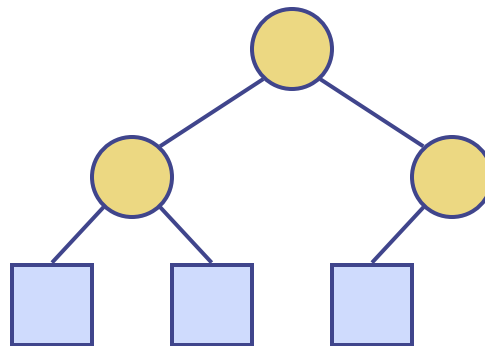


二叉树的性质

- 高度为 h 且有 $2^{(h+1)} - 1$ 个节点的二叉树，称为**完美二叉树**（perfect binary tree），或**满二叉树**
- 完全二叉树**（complete binary tree）：高度为 h 的完全二叉树，除第 h 层外其他各层结点数均达到最大值 2^i ；第 h 层元素尽可能存放在左侧的节点中，使得若某节点右子树高度为 l ，则其左子树高度为 l 或 $l + 1$



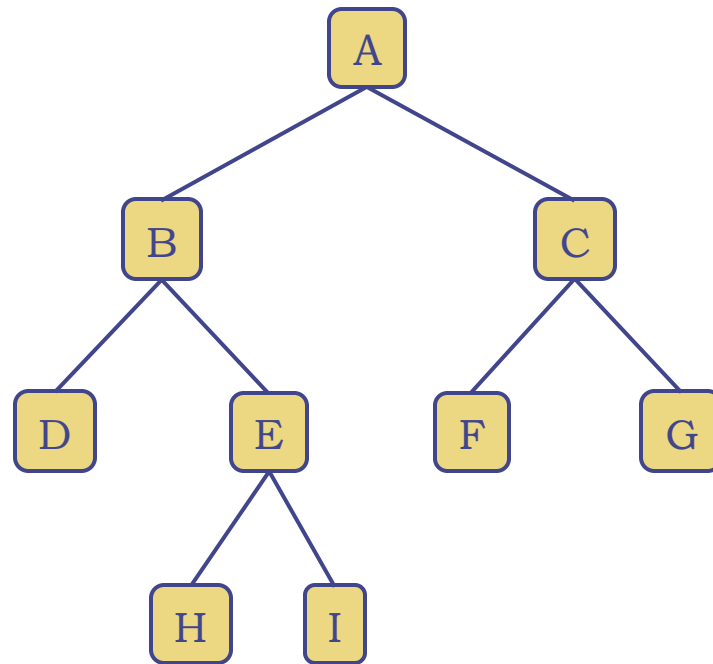
完美二叉树



完全二叉树

二叉树的性质

- 完满二叉树 (full binary tree) : 也称为proper binary tree, 所有内部节点度均为2 (均有两个孩子) 的二叉树



二叉树的ADT

- 二叉树的ADT是对树的ADT的拓展，它继承了树的ADT的所有基本操作，并附加了以下操作：
 - `T.left(p)`: 返回`p`的左孩子节点，若没有则返回`None`
 - `T.right(p)`: 返回`p`的右孩子节点，若没有则返回`None`
 - `T.sibling(p)`: 返回`p`的兄弟节点，若没有则返回`None`
- 更新二叉树的方法在具体描述二叉树实现时再定义

二叉树的python实现

- 二叉树的部分方法的python实现:

```
def sibling(self, p):
    """Return a Node representing p's sibling (or None if no sibling)."""
    parent = self.parent(p)
    if parent is None:
        return None
    else:
        if p == self.left(parent):
            return self.right(parent)
        else:
            return self.left(parent)

def children(self, p):
    """Generate an iteration of Nodes representing p's children."""
    if self.left(p) is not None:
        yield self.left(p)
    if self.right(p) is not None:
        yield self.right(p)
```



Today's Agenda

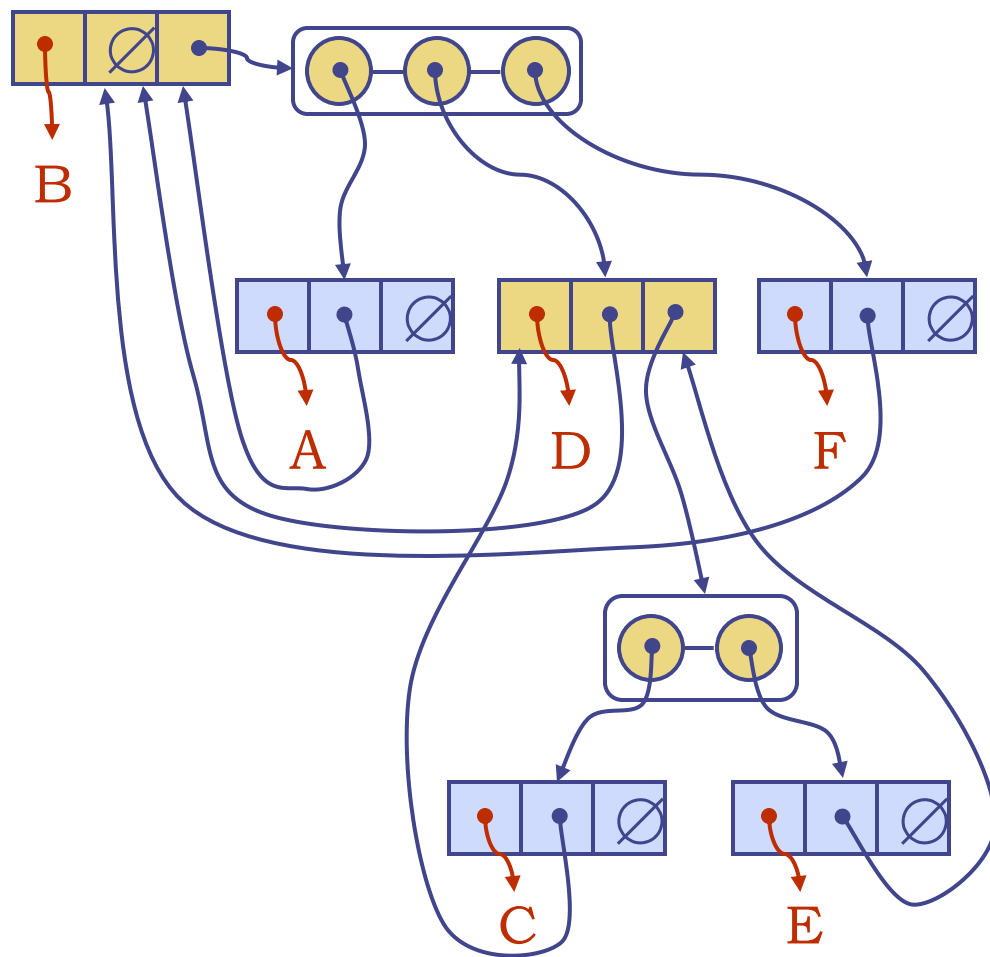
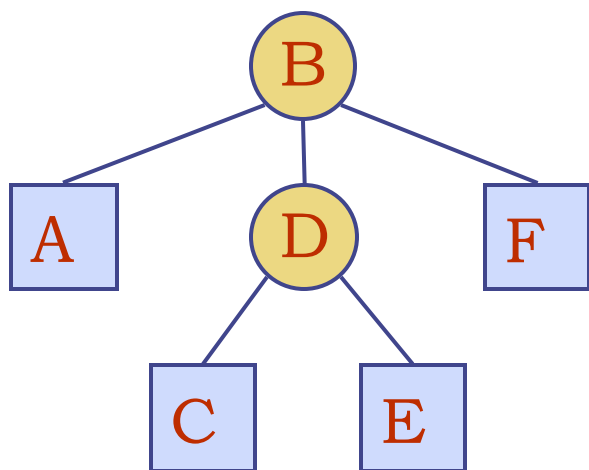
- 树
- 二叉树
- 树的实现
- 树的遍历算法

树的实现

- 目前为止，我们还没有实现树的root、parent、num_children、children、__len__等具体方法，以及对二叉树的left和right的具体方法
- 我们可以采用链式结构存储树，也可采用数组存储树
- 以下以二叉树为例介绍树的具体实现方式

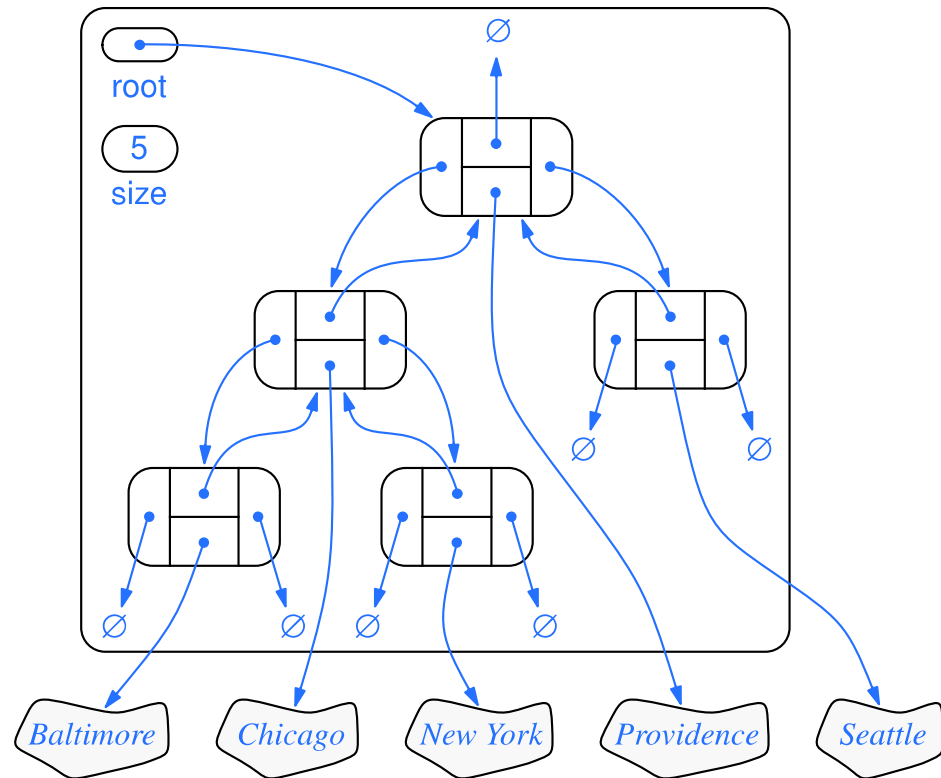
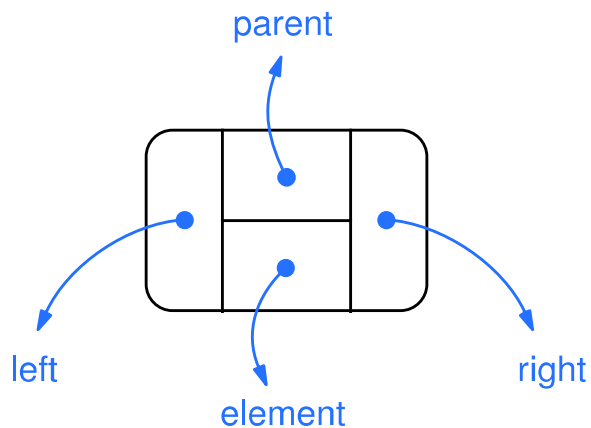
树的链式结构实现

- 一个节点存储以下信息：
 - 元素值
 - 指向父节点的指针
 - 指向存储其孩子节点的引用的指针



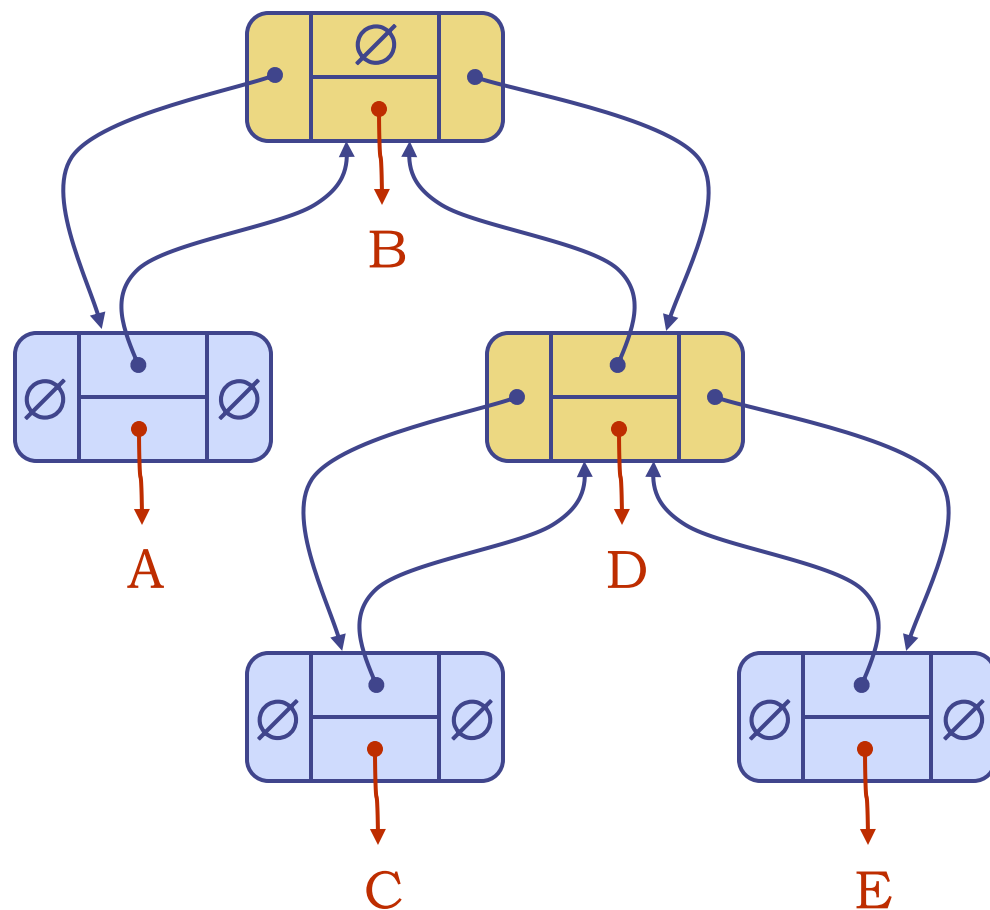
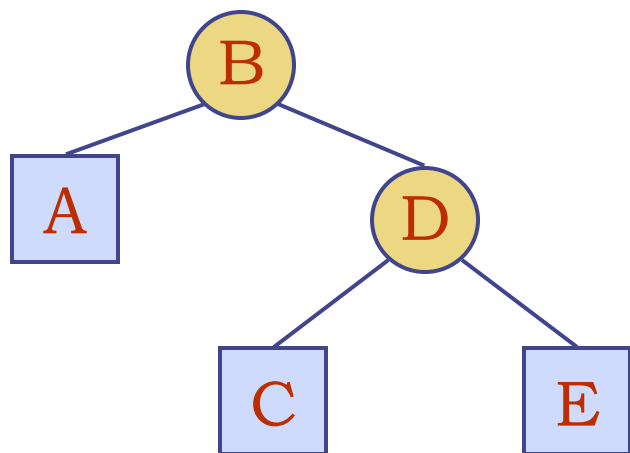
二叉树的链式结构实现

- 一个节点存储以下信息：
 - 元素值
 - 指向父节点的指针
 - 指向左孩子的指针
 - 指向右孩子的指针



二叉树的链式结构实现

- 例：将以下树用链式结构实现



链式二叉树的ADT

- 链式二叉树的更新方法如下：
 - `T.add_root(e)`: 为空树创建根节点，存储元素e，并返回根节点
 - `T.add_left(p, e)`: 为节点p创建新的左孩子节点存储元素e，返回左孩子节点；如p已有左孩子，则抛出错误
 - `T.add_right(p, e)`: 为节点p创建新的右孩子节点存储元素e，返回右孩子节点；如p已有右孩子，则抛出错误
 - `T.replace(p, e)`: 用元素e替代存储在节点p的元素，并返回之前存储的元素
 - `T.delete(p)`: 移除节点p，用它的孩子代替自己，并返回存储在p的元素值；若p有两个孩子，则抛出错误

链式二叉树的实现

```
class LinkedBinaryTree:
    """Linked representation of a binary tree structure."""

    #----- nested Node class -----
    class Node:
        """Lightweight, public class for storing a node."""
        __slots__ = '_element', '_parent', '_left', '_right' # streamline memory
        usage

        def __init__(self, element, parent=None, left=None, right=None):
            self._element = element
            self._parent = parent
            self._left = left
            self._right = right

        def element(self):
            """Return the element stored at this Node."""
            return self._element

        def __eq__(self, other):
            """Return True if other is a Node representing the same location."""
            return type(other) is type(self) and other is self
```

链式二叉树的实现

```
def __ne__(self, other):  
    """Return True if other does not represent the same location."""  
    return not (self == other)          # opposite of __eq__
```

```
#----- binary tree constructor -----
```

```
def __init__(self):  
    """Create an initially empty binary tree."""  
    self._root = None  
    self._size = 0
```

```
#----- public accessors -----
```

```
def __len__(self):  
    """Return the total number of elements in the tree."""  
    return self._size
```

```
def root(self):  
    """Return the root of the tree (or None if tree is empty)."""  
    return self._root
```

```
def parent(self, p):  
    """Return p's parent (or None if p is root)."""  
    return p._parent
```

链式二叉树的实现

```
def left(self, p):
    """Return p's left child (or None if no left child)."""
    return p._left

def right(self, p):
    """Return p's right child (or None if no right child)."""
    return p._right

def num_children(self, p):
    """Return the number of children of p."""
    count = 0
    if p._left is not None:      # left child exists
        count += 1
    if p._right is not None:     # right child exists
        count += 1
    return count
```

链式二叉树的实现

```
#----- nonpublic mutators -----
def _add_root(self, e):
    """Place element e at the root of an empty tree and return it.

    Raise ValueError if tree nonempty.
    """
    if self._root is not None:
        raise ValueError('Root exists')
    self._size = 1
    self._root = self.Node(e)
    return self._root

def _add_left(self, p, e):
    """Create a new left child for p, storing element e.

    Return the new node.
    Raise ValueError if p is invalid or p already has a left child.
    """
    if p._left is not None:
        raise ValueError('Left child exists')
    self._size += 1
    p._left = self.Node(e, p)                # p is its parent
    return p._left
```

链式二叉树的实现

```
def _add_right(self, p, e):  
    """Create a new right child for p, storing element e.  
  
    Return the new node.  
    Raise ValueError if p is invalid or p already has a right child.  
    """  
    if p._right is not None:  
        raise ValueError('Right child exists')  
    self._size += 1  
    p._right = self._Node(e, p)           # p is its parent  
    return p._right  
  
def _replace(self, p, e):  
    """Replace the element at node p with e, and return old element."""  
    old = p._element  
    p._element = e  
    return old
```

链式二叉树的实现

```
def _delete(self, p):
    """Delete the node p, and replace it with its child, if any.

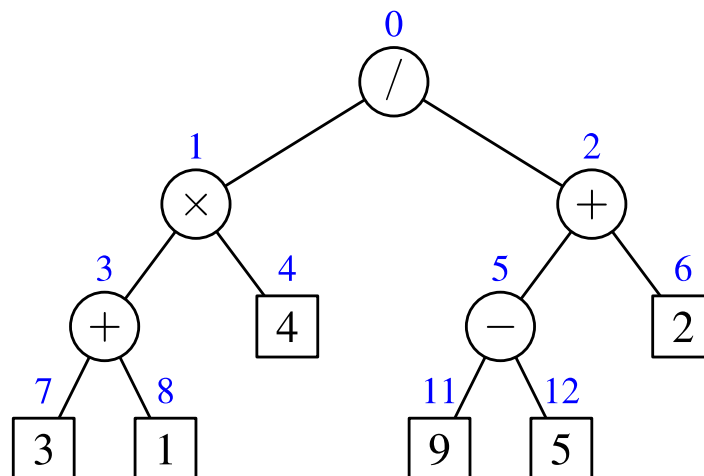
    Return the element that had been stored at p.
    Raise ValueError if p is invalid or p has two children.
    """
    if self.num_children(p) == 2:
        raise ValueError('Node has two children')
    child = p._left if p._left else p._right # might be None
    if child is not None:
        child._parent = p._parent           # child's grandparent becomes parent
    if p is self._root:
        self._root = child                  # child becomes root
    else:
        parent = p._parent
        if p is parent._left:
            parent._left = child
        else:
            parent._right = child
    self._size -= 1
    p._parent = p                            # convention for deprecated node
    return p._element
```

二叉树的数组实现

- 还可以使用数组存储二叉树
- 需要对每个节点 p 进行编号：
 - 若 p 是根节点，则 $f(p) = 0$
 - 若 p 是 q 的左孩子，则 $f(p) = 2f(q) + 1$
 - 若 p 是 q 的右孩子，则 $f(p) = 2f(q) + 2$
- 编号函数 f 称为树的位置的层编号
- p 的父节点的编号为 $\left\lfloor \frac{f(p)-1}{2} \right\rfloor$

二叉树的数组实现

- 一个例子：

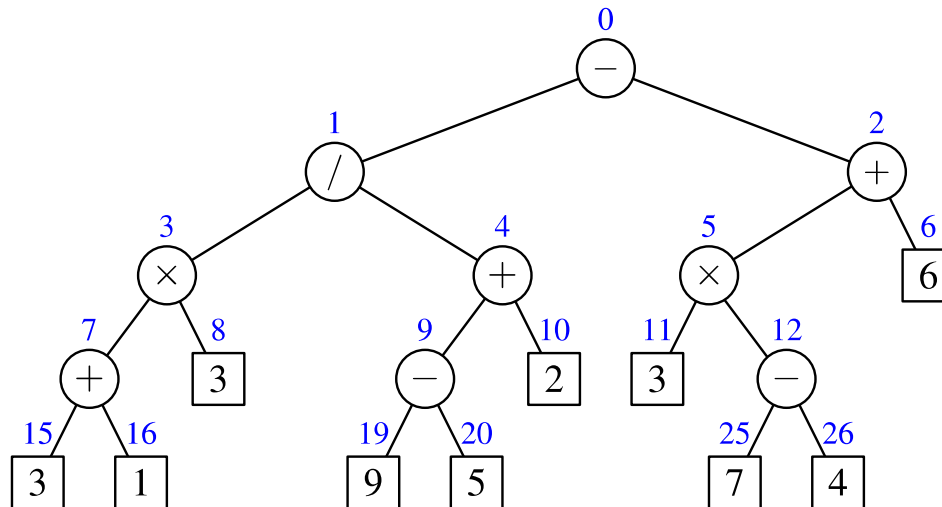


- 其对应的数组为：

/	×	+	+	4	-	2	3	1			9	5		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

二叉树的数组实现

- 另一个例子:



二叉树的数组实现

- 数组实现二叉树的缺点：
 - 空间使用极大依赖于树的形状，在二叉树为完全二叉树时效率最高，最坏情况下数组长度 N 与节点数 n 有 $N = 2^n - 1$ 的关系
 - 删除、插入等更新操作花费时间最坏为 $O(n)$

ANY
QUESTIONS
?

www.rekrutitn.com

ReKrutitn.com

Today's Agenda

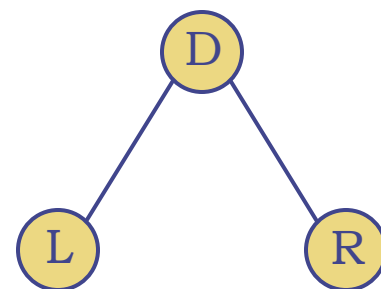
- 树
- 二叉树
- 树的实现
- 树的遍历算法

树的遍历算法

- 树的**遍历** (traversal) : 按某种规则不重复地访问树的所有节点
- **前序遍历** (preorder traversal) : 先访问根, 再递归访问子树。如树有序, 则按顺序访问子树
- **后序遍历** (postorder traversal) : 先递归访问子树, 再访问根。如树有序, 则按顺序访问子树
- 二叉树的**中序遍历** (inorder traversal) : 先递归访问左孩子, 再访问根节点, 再递归访问右孩子
- **层序遍历/广度优先遍历** (level order traversal/breadth-first traversal) : 按层为顺序遍历, 每一层从左到右遍历

树的遍历算法

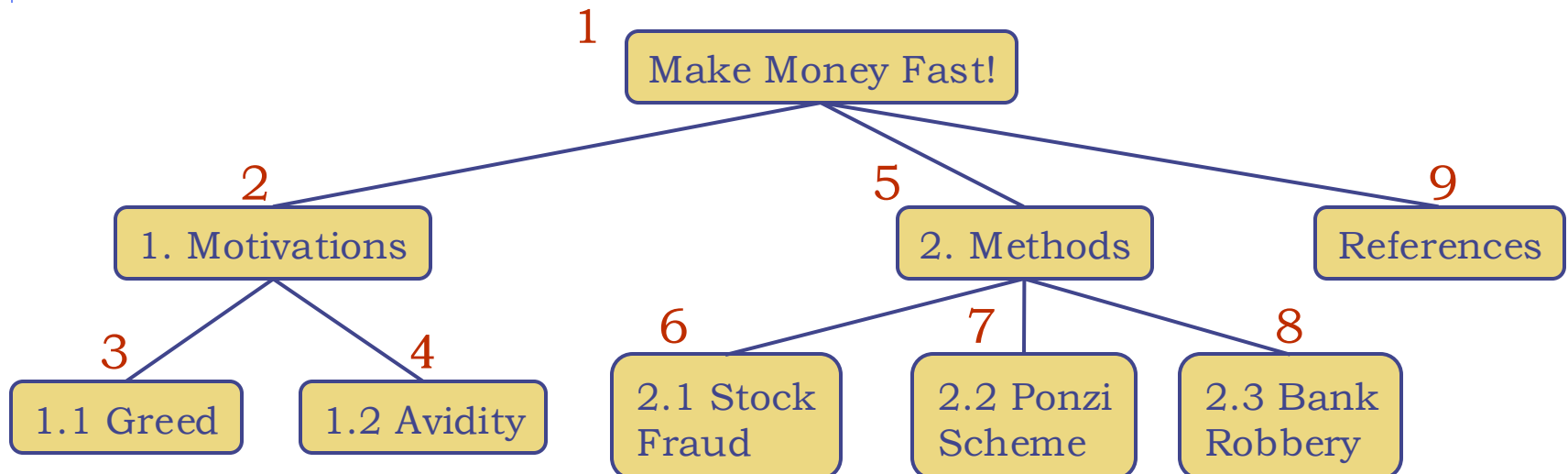
- 二叉树的遍历算法：
 - 设二叉树由根 D 、左子树 L 和右子树 R 构成
 - 前序遍历： $D \rightarrow L \rightarrow R$
 - 后序遍历： $L \rightarrow R \rightarrow D$
 - 中序遍历： $L \rightarrow D \rightarrow R$
 - 层序遍历：从低到高逐层从左到右遍历



前序遍历

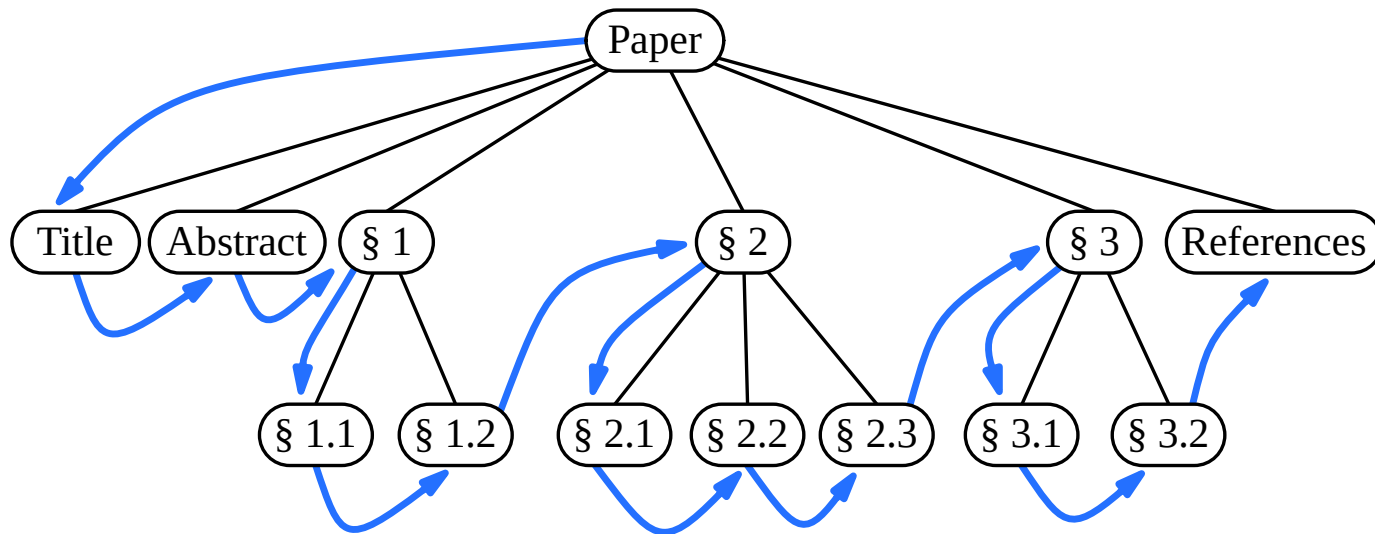
- 访问根节点
- 前序遍历左子树
- 前序遍历右子树
- 例:

```
Algorithm preOrder(T, p):  
    visit(p)  
    for each child c in T.children(p) do  
        preOrder(T, c)
```



前序遍历

- 例：打印一个有层次结构的文档



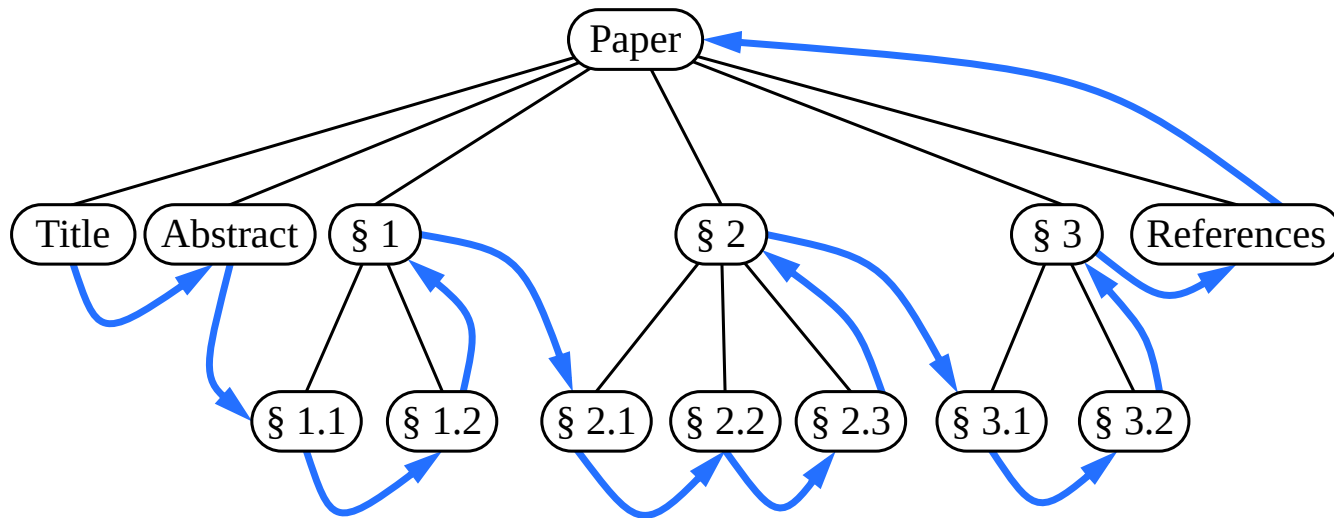
前序遍历的实现

```
def _subtree_preorder(self, p):  
    """Generate a preorder iteration of nodes in subtree rooted at p."""  
    yield p                                # visit p before its subtrees  
    for c in self.children(p):            # for each child c  
        for other in self._subtree_preorder(c): # do preorder of c's subtree  
            yield other                      # yielding each to our caller  
  
def preorder(self):  
    """Generate a preorder iteration of nodes in the tree."""  
    if not self.is_empty():  
        for p in self._subtree_preorder(self.root()): # start recursion  
            yield p
```

后序遍历

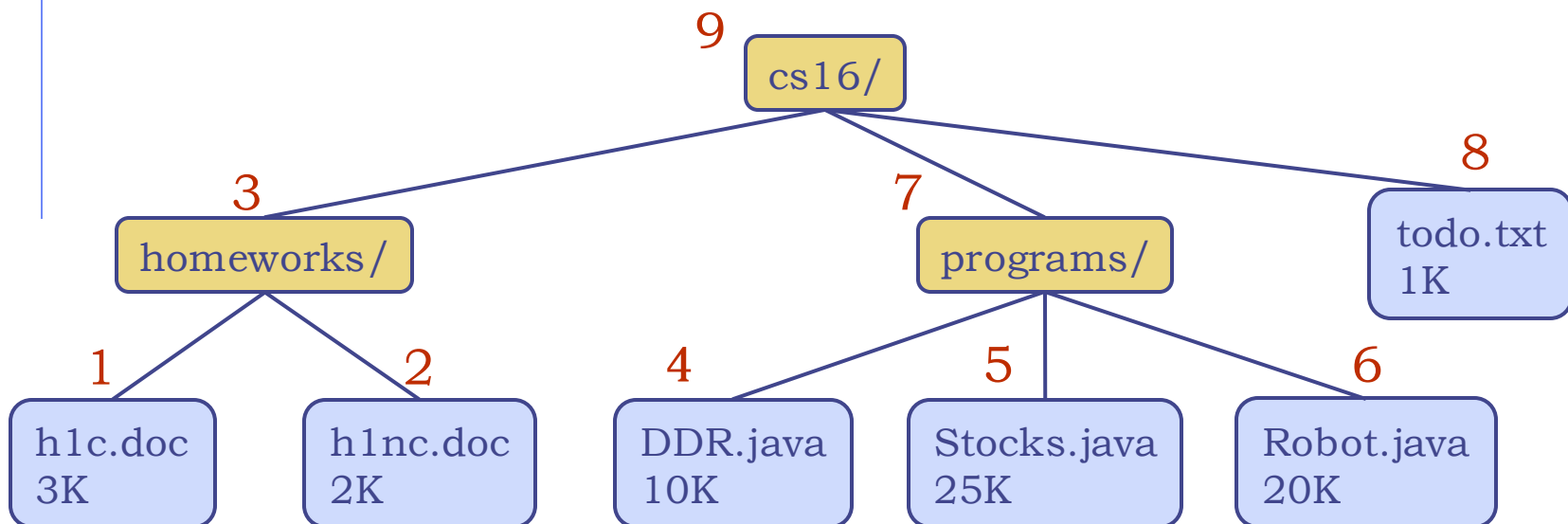
- 后序遍历左子树
- 后序遍历右子树
- 访问根节点
- 例：

```
Algorithm postOrder(T, p):  
    for each child c in T.children(p) do  
        postOrder(T, c)  
    visit(p)
```



后序遍历

- 例：计算文件夹存储空间



后序遍历的实现

```
def _subtree_postorder(self, p):  
    """Generate a postorder iteration of nodes in subtree rooted at p."""  
    for c in self.children(p):  
        # for each child c  
        for other in self._subtree_postorder(c):  
            # do postorder of c's subtree  
            yield other  
        # yielding each to our caller  
    yield p  
    # visit p after its subtrees  
  
def postorder(self):  
    """Generate a postorder iteration of nodes in the tree."""  
    if not self.is_empty():  
        for p in self._subtree_postorder(self.root()): # start recursion  
            yield p
```

二叉树的中序遍历

- 中序遍历左子树
- 访问根节点
- 中序遍历右子树
- 例:

Algorithm *inOrder*(*T*, *p*):

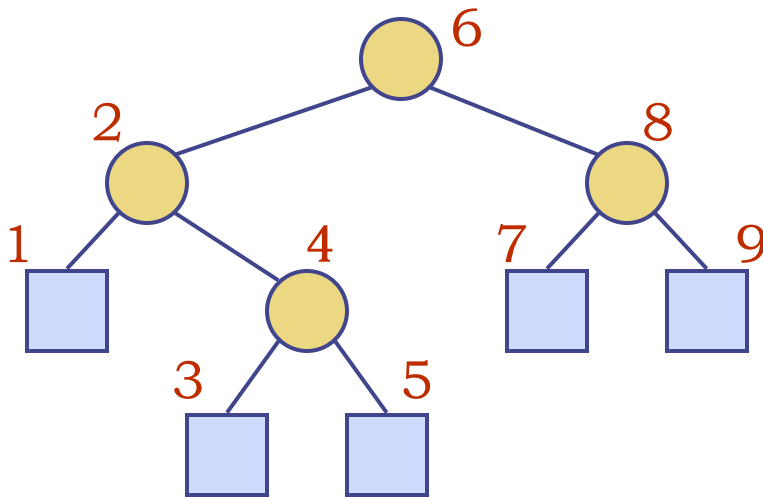
if *p* has a left child *lc* **then**

inOrder(*T*, *lc*)

visit(*p*)

if *p* has a right child *rc* **then**

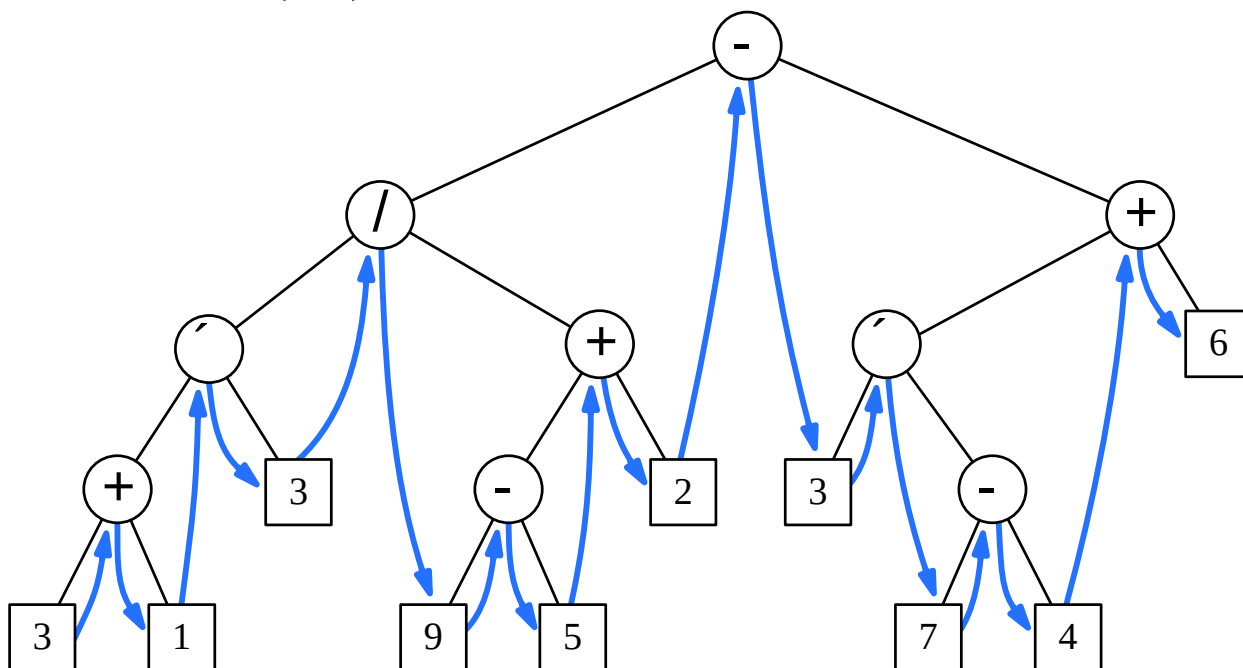
inOrder(*T*, *rc*)



二叉树的中序遍历

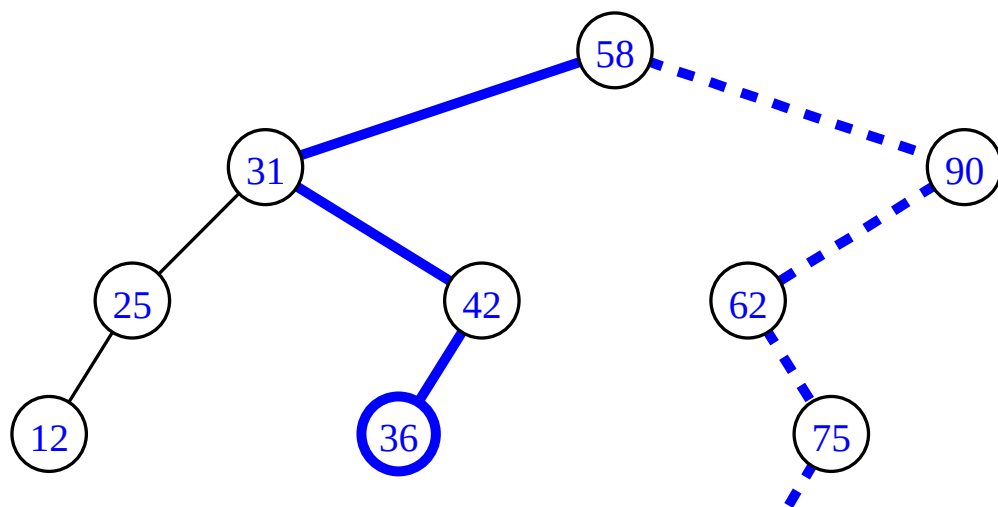
· 例：表达式树

· 以下树表示 $\frac{(3+1) \times 3}{(9-5)+2} - (3 \times (7-4) + 6)$



二叉树的中序遍历

- 例：二叉搜索树
 - 二叉搜索树的中序遍历即表示其所存储的有序序列的顺序



二叉树的中序遍历的实现

```
def _subtree_inorder(self, p):  
    """Generate an inorder iteration of nodes in subtree rooted at p."""  
    if self.left(p) is not None:    # if left child exists, traverse its subtree  
        for other in self._subtree_inorder(self.left(p)):  
            yield other  
    yield p                        # visit p between its subtrees  
    if self.right(p) is not None:   # if right child exists, traverse its subtree  
        for other in self._subtree_inorder(self.right(p)):  
            yield other  
  
def inorder(self):  
    """Generate an inorder iteration of nodes in the tree."""  
    if not self.is_empty():  
        for p in self._subtree_inorder(self.root()):  
            yield p
```

树的遍历算法

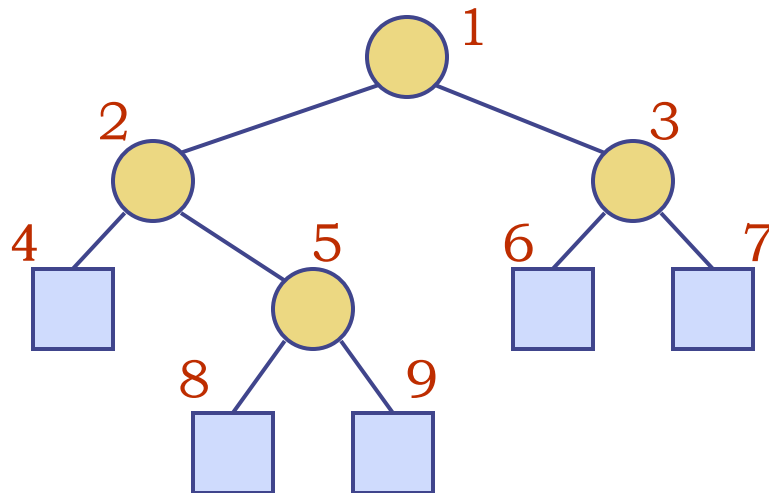
- 前/中/后序遍历均为**深度优先遍历**算法，可用递归实现，也可用模拟递归栈的方式进行非递归实现
- 递归栈的最大深度和树的深度保持一致
 - 最好情况，空间复杂度 $O(\log n)$
 - 最坏情况，空间复杂度 $O(n)$
- 我们还可以使用**广度优先遍历**算法，即**层序遍历**算法

树的层序遍历

- 按照层次自低向高，每层从左向右访问
- 无法用递归实现，借助队列：
 - 先将根节点入队
 - 每出队一个节点，将其孩子节点依次放入队列
- 例：

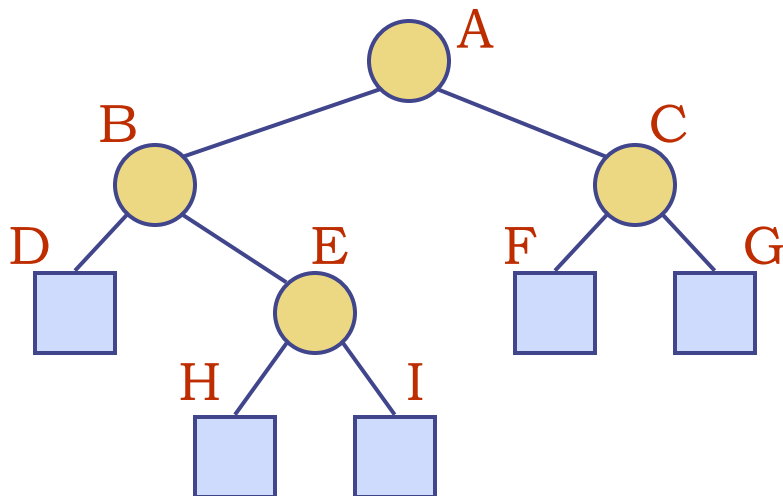
Algorithm *breadthfirst*(*T*):

```
Initialize queue Q to contain T.root()  
while Q not empty do  
    p = Q.dequeue()  
    visit(p)  
    for each child c in T.children(p) do  
        Q.enqueue(c)
```



树的层序遍历

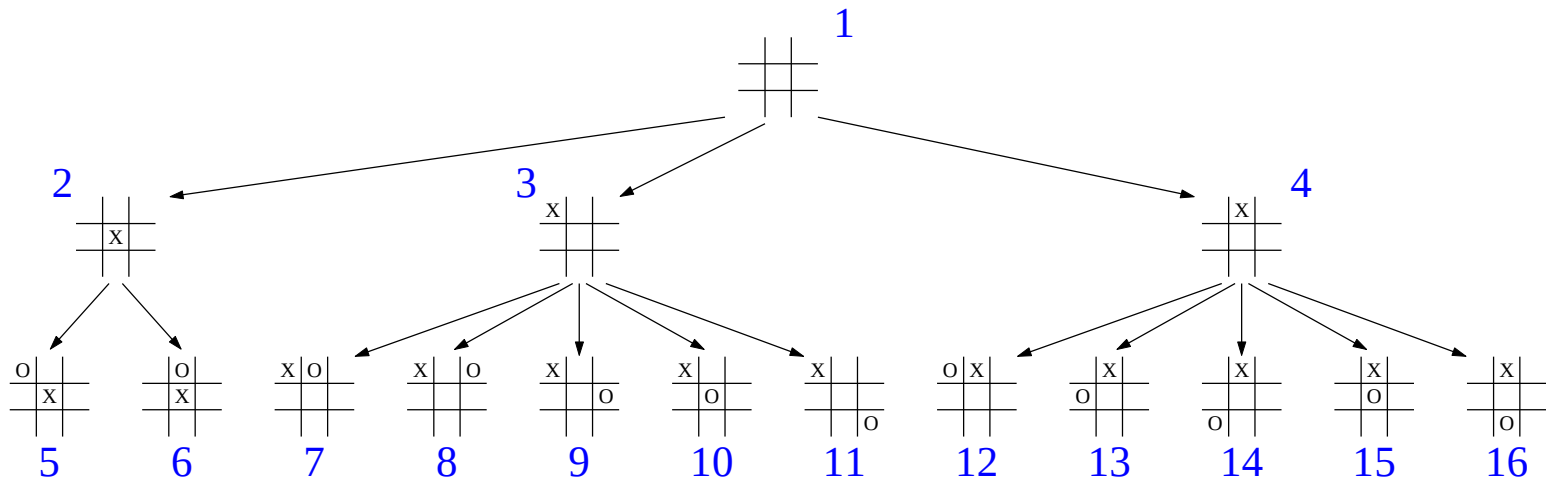
- 树的层序遍历中队列的变化：



A				
B	C			visit(A)
C	D	E		visit(B)
D	E	F	G	visit(C)
E	F	G		visit(D)
F	G	H	I	visit(E)
G	H	I		visit(F)
H	I			visit(G)
I				visit(H)
				visit(I)

树的层序遍历

- 例：井字棋的部分博弈树
 - 博弈树用作表示玩家下一步可选择的一些动作



树的层序遍历的实现

```
def breadthfirst(self):  
    """Generate a breadth-first iteration of the nodes of the tree."""  
    if not self.is_empty():  
        fringe = LinkedQueue()           # known positions not yet yielded  
        fringe.enqueue(self.root())      # starting with the root  
        while not fringe.is_empty():  
            p = fringe.dequeue()          # remove from front of the queue  
            yield p                       # report this position  
            for c in self.children(p):  
                fringe.enqueue(c)         # add children to back of queue
```



树遍历的应用

- 目录表:
 - 树的**前序遍历**可以自然地用于产生文档或书籍的目录表
 - 如不需要缩进（如左图），则可直接使用如下代码打印目录表：

```
for p in T.preorder():  
    print(p.element())
```

- 如需要缩进（如右图），则需要定义一个特殊的前序遍历函数

```
Paper  
Title  
Abstract  
§1  
§1.1  
§1.2  
§2  
§2.1  
...
```

```
Paper  
    Title  
    Abstract  
    §1  
        §1.1  
        §1.2  
    §2  
        §2.1  
    ...
```

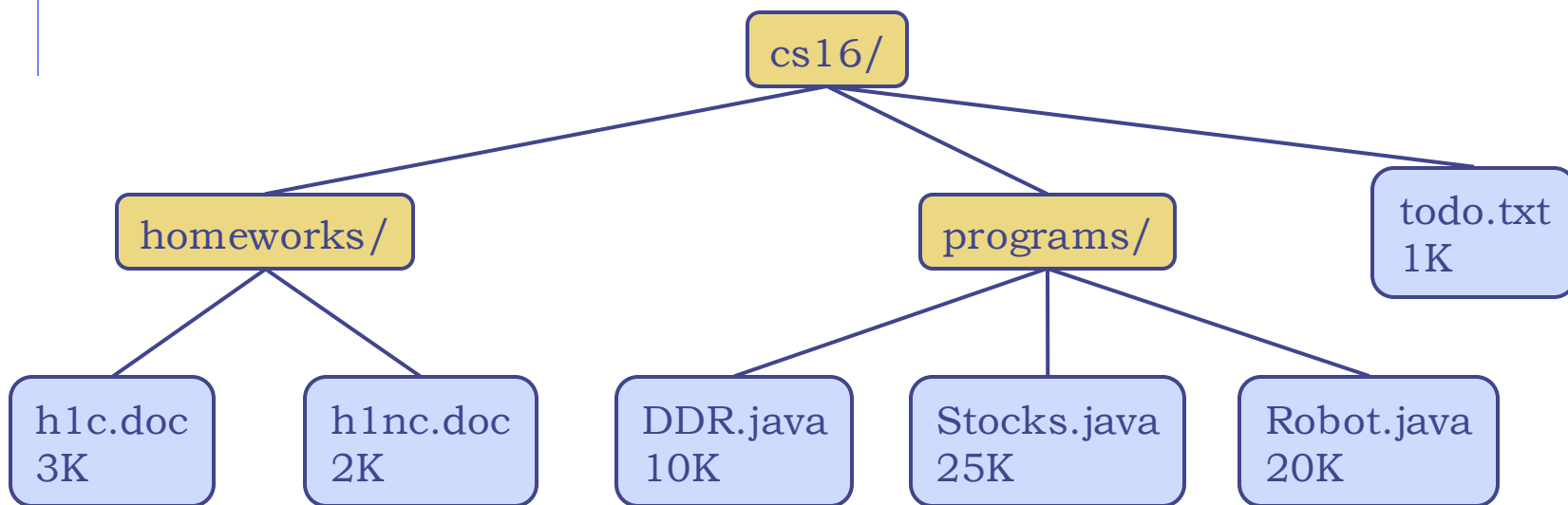
树遍历的应用

- 目录表:
 - 如需要缩进（如右图），则需要定义一个特殊的前序遍历函数：

```
def preorder_indent(T, p, d):  
    """Print preorder representation of subtree of T rooted at p at depth d."""  
    print(2*d*' ' + str(p.element()))          # use depth for indentation  
    for c in T.children(p):  
        preorder_indent(T, c, d+1)              # child depth is d+1
```

树遍历的应用

- 计算磁盘空间：
 - 计算磁盘空间需要将文件系统表示为树后，使用**后序遍历**
 - 需要定义一个特殊的后序遍历函数记录当前占有的存储空间



树遍历的应用

- 计算磁盘空间：
 - 需要定义一个特殊的后序遍历函数记录当前占有的存储空间

```
def disk_space(T, p):  
    """Return total disk space for subtree of T rooted at p."""  
    subtotal = p.element().space()           # space used at position p  
    for c in T.children(p):  
        subtotal += disk_space(T, c)         # add child's space to subtotal  
    return subtotal
```

树的应用：表达式树

- 表达式树是一棵二叉树
- 使用表达式树输出表达式需要一种特殊的**中序遍历**算法
 - 访问节点时输出节点存储的值或运算符
 - 遍历左子树前输出 “(”
 - 遍历右子树后输出 “)”

Algorithm *printExpression(v)*:

if *v* **has a left child**

print("(")

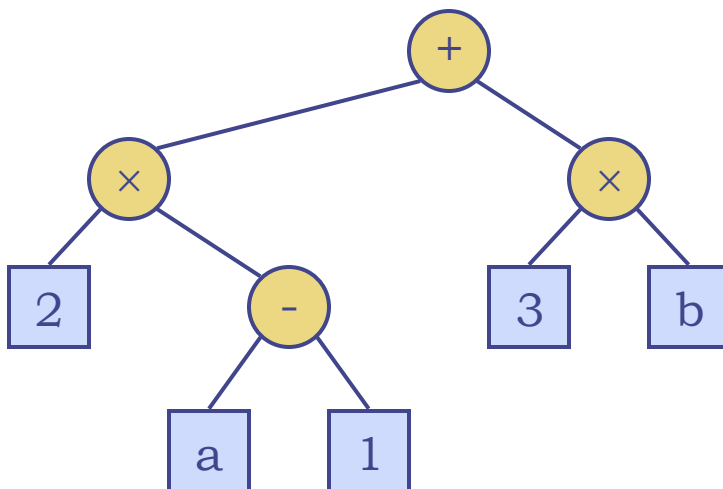
printExpression(*left(v)*)

print(*v.element*())

if *v* **has a right child**

printExpression(*right(v)*)

print (")")



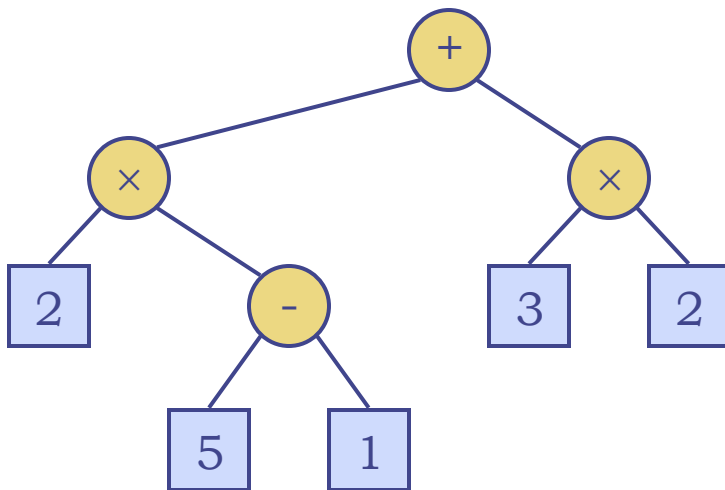
$((2 \times (a - 1)) + (3 \times b))$

树的应用：表达式树

- 使用表达式树计算表达式的值需要一种特殊的**后序遍历**
 - 使用递归返回子树的值
 - 访问内部节点时，使用内部节点的运算符对左、右子树的值做运算

Algorithm *evalExpr(v)*:

```
if is_leaf (v)
    return v.element ()
else
     $x \leftarrow evalExpr(left(v))$ 
     $y \leftarrow evalExpr(right(v))$ 
     $\diamond \leftarrow$  operator stored at v
    return  $x \diamond y$ 
```





Enjoy